

GRIMOIRE VAULT · V1.0.0

Upgrading

Гайд обновления — миграции, версии, breaking changes

Upgrading

Что делать, когда обновляешь версию приложения, тянешь свежий код, меняешь зависимости или мигрируешь схему БД.

Версионирование

Проект использует **semantic versioning** (`MAJOR.MINOR.PATCH`). Текущая версия — в `package.json` → `"version"`.

Что попадает в каждый разряд:

- **MAJOR** — breaking change в API или схеме БД, требующий миграции данных или обновления клиента (например, изменение структуры `export payload version` с 1 на 2).
- **MINOR** — новая фича, обратная совместимость.
- **PATCH** — баг-фикс, оптимизация.

Версия отображается в:

- `/api/health` (public)
- `/settings` → `Vault stats` (owner-only)
- `/admin/health` page header

Если на проде висит не та версия, что в `package.json` → деплой не прошёл, проверь Vercel.

Миграции БД — runner

```
# Покажи pending миграции, ничего не применяй:
npm run migrate:plan

# Применить все pending в порядке имени файла:
npm run migrate
```

Что под капотом

Скрипт `scripts/migrate.mjs`:

1. Считывает все файлы из `supabase/migrations/*.sql`.
2. Запрашивает у Postgres `SELECT name FROM schema_migrations`.
3. Diff'ит → находит pending.
4. Для каждой pending:
 - Применяет через Supabase Management API.
 - Записывает строку в `schema_migrations` с timestamp.

Требуемые env vars

GRIMOIRE VAULT · UPGRADING

```
SUPABASE_PROJECT_REF=xxxxxxxxxxxxxxxxxxxxx
SUPABASE_ACCESS_TOKEN=sbp...
```

`SUPABASE_PROJECT_REF` — короткий ID проекта (виден в URL Supabase Dashboard).

`SUPABASE_ACCESS_TOKEN` — Personal Access Token из Supabase Account → Access Tokens.

 *Никогда не коммить эти переменные. Они дают права на любые операции в твоём Supabase-аккаунте.*

Идемпотентность

Каждая миграция должна быть безопасно-перезапускаемой. Используй:

- `CREATE TABLE IF NOT EXISTS`
- `CREATE INDEX IF NOT EXISTS`
- `DROP X IF EXISTS` перед `CREATE`
- `INSERT ... ON CONFLICT DO NOTHING`
- `ALTER TABLE x ADD COLUMN IF NOT EXISTS`

Если миграция один раз сломалась посередине, повторное применение должно довести БД до целевого состояния.

Что делать, если применил миграцию вручную через SQL Editor

`schema_migrations` не узнает об этом → runner попытается переприменить. Решение: добавь строку вручную:

```
insert into public.schema_migrations (name, applied_by)
values ('20260504080000_my_migration', 'manual')
on conflict (name) do nothing;
```

Или сбрось весь лог из имён файлов:

```
node scripts/migrate.mjs --reset-log
```

(Используй с осторожностью — это говорит «всё, что в `supabase/migrations/`, считай применённым».)

Обновление зависимостей

Обычный bump

```
npm outdated      # увидеть, что устарело
npm update         # bump в пределах semver-range из package.json
npm run typecheck  # tsc --noEmit
npm run lint
npm run build      # next build
npm run test       # playwright
```

Если все проверки зелёные — деплой.

Major bump (Next.js / React / Supabase)

1. Прочти их CHANGELOG / migration guide.
2. `npm install <pkg>@latest` для одной зависимости за раз.
3. `npm run typecheck` → исправь breaking type changes.
4. `npm run build` → исправь breaking runtime.
5. Прогоните **обе** verification suite:

```
npm run verify:api
npm run test
```

6. Только после этого деплой.

Что было обновлено в этом проекте

ЗАВИСИМОСТЬ	ТЕКУЩАЯ ВЕРСИЯ	ЗАМЕТКИ
Next.js	16.2.4	App Router, RSC, Turbopack
React	19.2.4	Server Components default
Supabase JS	2.105.1	client + ssr + supabase-js
<code>@huggingface/transformers</code>	4.2.0	Browser-only, lazy-loaded
Tailwind	4.x	Без отдельного config файла
TypeScript	5.x	strict mode

Обновление Postgres / Supabase

Supabase автоматически обновляет:

- **Minor versions** Postgres — без действий с твоей стороны.
- **Patch updates** расширений — также автоматически.

Major Postgres upgrade (например, 17 → 18) — Supabase предупредит заранее в Dashboard.
Действия:

1. Сделай Full Export (Settings → Export Vault).
2. Прочти migration notes от Supabase.
3. В случае проблем — восстанавливай из backup’а.

Обновление API contract

Если меняется shape API-ответа (`POST /api/entries` теперь возвращает другие поля), это breaking для клиентов на старых build’ах в браузере у пользователей.

Что делать

1. **Деприкируй**, не удаляй: пока клиенты используют старое поле, возвращай и старое, и новое одновременно.
2. **Bump MINOR** при добавлении нового опционального поля.
3. **Bump MAJOR** при удалении или смене семантики существующего.

4. Service Worker автоматически обновится после первого запроса (cache-first для `_next/static/*`, network-first для pages).

GRIMOIRE VAULT · UPGRADING

Export schema versioning

`/api/export` возвращает `version: 1`. При breaking change в формате дампа:

1. Bump version field до `2`.
2. `/api/import` должен принимать **обе** версии и upgrade'ить старую при импорте.
3. Schema валидация в `lib/schemas/import.ts` принимает union.

Service Worker version

Когда меняешь `public/sw.js`, **обязательно** bump'ни константу `VERSION = "vX.Y.Z"`. Без этого старые SW в браузерах юзеров не заметят обновления.

Принцип

- При изменении `VERSION` старые кэши инвалидируются.
- Юзеры получают новый SW при следующей navigation request.
- Никаких force-refresh не нужно — Service Worker умеет сам.

Изменения в env vars

При добавлении нового env var:

1. Добавь в `.env.example` с комментарием — что это, обязательное или нет.
2. В коде используй `process.env.X ?? sensible_default` или fail-closed (если переменная критична — `OWNER_EMAIL`, `VAPID_PRIVATE_KEY`).
3. Обнови `.env.example` PR-ом, не отдельно.
4. На Vercel — `vercel env add NEW_VAR production --force`.

Если переменная вдруг сменилась

`/admin/health` пробит зависимости — если что-то сломалось от смены env, ты увидишь красную плитку. Запускай после **каждого** деплоя.

Регенерация PDF документации

После любого изменения в `docs/*.md`:

```
npm run docs:pdf
```

Это регенерирует все PDF в `docs/pdf/` через Playwright (рендер markdown → HTML → PDF). PDF'ы коммитаются в репо для пользователей, у которых нет markdown-aware reader'a.

Не правь PDF вручную — они полностью генерируются из MD.

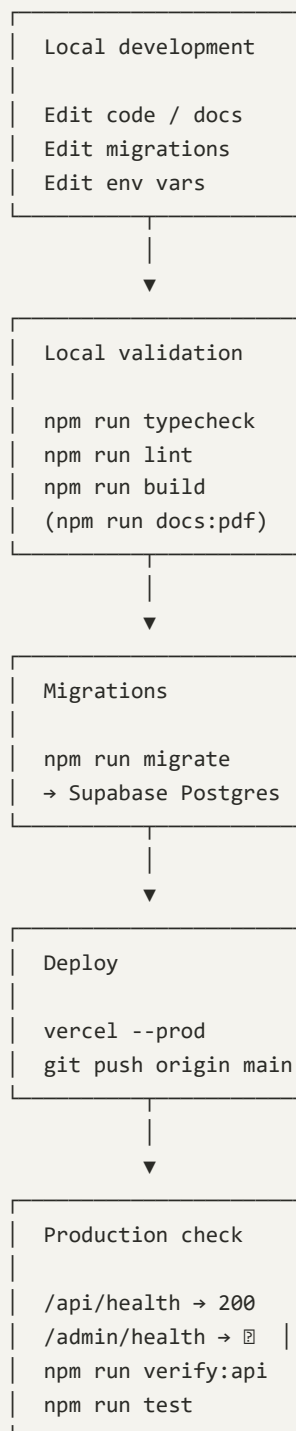
Чек-лист «обновил, что дальше»

После `git pull`:

- [] `npm install` — подтянуть новые зависимости.
- [] `npm run migrate:plan` — увидеть pending миграции (если есть).
- [] `npm run migrate` — применить.
- [] `npm run typecheck` — strict-проверка.
- [] `npm run lint`.
- [] `npm run build`.
- [] Если меняли env vars → синхронизировать `.env.local` с `.env.example`.
- [] `npm run verify:api` (опц.) — прогнать API-ассерты против локального дев-сервера или прода.
- [] `npm run test` (опц.) — Playwright против прода.
- [] `npm run dev` — поднять локально, прокликать.

После деплоя:

- [] Открыть `/admin/health` → 5 зелёных плиток.
 - [] Открыть `/api/health` → проверить, что версия совпадает с `package.json`.
 - [] Открыть `/settings → Vault stats` → проверить migrations counter и runtime info.
-



Обновляется вместе с *migrate runner*'ом и стеком. Если описанный workflow перестал соответствовать реальности — патч сюда первой очередью.